# STUDENT INFORMATION SYSTEM

**A PROJECT REPORT**

***Submitted by***


| | |
|---|---|
| **JAFFIN DURAI.T** | **961321104011** |
| **SHIJIN KUMAR.D** | **961321104022** |
| **SUMAN.G** | **961321104025** |
| **VISHNU.V** | **961321104701** |


***in partial fulfillment for the award of the degree***

***of***

**BACHELOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**


**M.E.T ENGINEERING COLLEGE**

**CHENBAGARAMANPUTHOOR-629304**


**ANNA UNIVERSITY: CHENNAI – 600025**

**MAY 2025**

# ANNA UNIVERSITY : CHENNAI 600 025

## BONAFIDE CERTIFICATE

Certified that this project report **"STUDENT INFORMATION SYSTEM"** is the bonafide work of **"SUMAN.G (961321104025)"** who carried out the project work under my supervision.

| | |
|---|---|
| **SIGNATURE** | **SIGNATURE** |
| Mrs. A. AUSLIN RANI MARY M.E | Mrs. A. ARTHIKA SUJI M.E |
| **HEAD OF THE DEPARTMENT** | **SUPERVISOR** |
| Department of CSE | Department of CSE |
| M.E.T Engineering College | M.E.T Engineering College |
| Chenbagaramanputhoor 629304 | Chenbagaramanputhoor 629304 |

Submitted for the project viva voice examination held on ……

INTERNAL EXAMINER EXTERNAL EXAMINER

## ACKNOWLEDGEMENT

First and foremost, we acknowledge the abiding presence and the abounding grace of our almighty god for his unseen hand yet tangible guidance all through the formation of this project. Also, we express our cordial thanks to our beloved parents.

We express our sincere gratitude to our Chairman **Er.P.MOHAMED EAKIEEM** for providing us all supports.

We are extremely grateful to our Chief Executive Officer **Dr.G.MAHADEVAN M.E,PhD** for his inspiration to us for preceding this project.

Our heartfelt thanks to our Principal **Mrs.G.B.EZHIL JENEKKHA M.E** for her constant help, affectionate support, suggestions and valuable ideas in conducting my study.

We would like to extend our heartfelt thanks to **Mrs.A.AUSLIN RANI MARY M.E** Head of Computer Science and Engineering Department for granting us permission and for providing a good working environment to complete this project work.

We express our heartiest thanks to all staffs of Computer Science and Engineering department who have helped us in for the successful completion of the project.

## ABSTRACT

The Student Information System is designed to provide students and teachers with seamless,hands-free access to their academic and professional information through a web-based interface. This system centralizes and manages comprehensive details, allowing users to add, update, and retrieve essential records efficiently.

Students can view their arrears, current CGPA, attendance percentage, submitted assignments, and other academic details. Teachers can view their class and subject allocations. Additionally, both students and teachers can download subject resources and view their timetables.

To enhance functionality, the system integrates online dues payment, enabling students to conveniently pay fees and other financial obligations. Furthermore, QR code authentication is implemented to provide a secure and effortless login experience.

This project serves as a valuable digital solution for educational institutions, streamlining administrative processes and improving accessibility for students and faculty.

## TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | **iv** |
| | **LIST OF FIGURES** | **viii** |
| | **LIST OF ABBREVIATIONS** | **ix** |
| **1** | **INTRODUCTION** | **1** |
| | 1.1 Overview of Student Information Systems | 1 |
| | 1.2 Importance of SIS in Educational Institutions | 2 |
| | 1.3 Project Objectives | 3 |
| | 1.4 Scope of the Project | 4 |
| **2** | **LITERATURE SURVEY** | **5** |
| | 2.1 Existing Systems and Solutions | 5 |
| | 2.2 Comparative Analysis of SIS Platforms | 6 |
| | 2.3 Research Gap and Opportunities | 7 |
| **3** | **BACKGROUND AND RELATED WORK** | **8** |
| | 3.1 Evolution of SIS Technologies | 8 |
| | 3.2 Key Features and Functionalities in SIS | 9 |
| | 3.3 Role of Automation and Integration in SIS | 10 |

**4 TECHNOLOGY STACK 11**

4.1 Overview of Technologies Used (MERN Stack) 11

4.2 Frameworks, Libraries, and Tools 12

4.3 Hardware Requirements 14

4.4 Software Requirements 14

**5 DESIGN AND ARCHITECTURE 15**

5.1 System Architecture Overview 15

5.2 Client-Server Architecture with MERN Stack 16

5.3 Backend Services and Database Interaction 17

5.4 Authentication and Security 17

5.5 Data Flow and Process Flow Diagrams 19

5.6 User Interface and Navigation Design 21

**6 MODULE IMPLEMENTATION 23**

6.1 Student Details Management 23

6.2 Marks and Grading Module 23

6.3 Fee Management and Payment Integration 24

6.4 Receipt Generation 25

6.4 QR Code Authentication 26

| | | |
|---|---|---|
| **7** | **TESTING AND VALIDATION** | **27** |
| | 7.1 Testing Strategy and Tools Used | 27 |
| | 7.2 Test Cases for Each Module | 27 |
| | 7.3 Security and Data Integrity Testing | 28 |
| **8** | **RESULTS AND ANALYSIS** | **29** |
| | 8.1 Performance Metrics | 29 |
| | 8.2 User Feedback and Usability | 30 |
| | 8.3 Data Accuracy and Reporting Quality | 30 |
| | 8.4 Overall System Evaluation | 31 |
| **9** | **CHALLENGES AND LIMITATIONS** | **32** |
| | 9.1 Technical Challenges | 32 |
| | 9.2 Operational and User-Related Issues | 32 |
| | 9.3 Limitations of the Current System | 33 |
| | **CONCLUSION** | **34** |
| | **FUTURE ENHANCEMENT** | **35** |
| | **APPENDICES** | **36** |
| | **REFERENCES** | **45** |

## LIST OF FIGURES


| FIGURE NO. | NAME OF FIGURE | PAGE NO. |
|---|---|---|
| 5.1 | System Architecture Diagram | 15 |
| 5.5 | Data Flow Diagram | 19 |
| 5.6 | User Interface and Navigation Design | 22 |

# LIST OF ABBREVIATIONS

| S. NO. | ABBREVIATION | EXPANSION |
|---|---|---|
| 1 | SIS | Student Information System |
| 2 | QR | Quick Response |
| 3 | UI | User Interface |
| 4 | UX | User Experience |
| 5 | ID | Identification |
| 6 | AI | Artifical Intelligence |
| 7 | CGPA | Cumulative Grade Point Average |
| 8 | JWT | Json Web Token |
| 9 | PDF | Portable Document Format |
| 10 | API | Application Programming Interface |
| 11 | MERN | MongoDB, Express.js, React, Node.js |
| 12 | HTTP | Hypertext Transfer Protocol |
| 13 | CSS | Cascading Style Sheets |
| 14 | S3 | Simple Storage Service |
| 15 | UPI | Unified Payments Interface |

| | | |
|---|---|---|
| 16 | DFD | Data Flow Diagram |
| 17 | JSON | JavaScript Object Notation |
| 18 | CRUD | Create, Read, Update, Delete |
| 19 | CSV | Comma Separated Values |
| 20 | ERP | Enterprise Resource Planning |

# CHAPTER – 1

# INTRODUCTION

## 1.1 Overview of Student Information Systems

A Student Information System (SIS) is a centralized platform designed to manage and streamline all academic, administrative, and financial information of students within an educational institution. These systems are typically web-based and serve as a core component in automating routine tasks, reducing manual effort, and improving data accessibility and accuracy.

SIS solutions provide functionalities such as student enrollment, attendance tracking, academic performance monitoring, timetable management, fee processing, and communication tools. These systems empower students to access their academic records, teachers to manage subject assignments and grades, and administrators to oversee overall operations. Role-based access controls ensure that each user has access to the appropriate resources, based on their role in the institution.

Modern SIS platforms are built with an emphasis on scalability, security, and user experience. They integrate advanced technologies such as cloud storage, digital payments, QR authentication, and responsive UI designs. This ensures that the system is robust, adaptable, and capable of handling large volumes of data and user interactions without compromising performance or security.

SIS solutions play a vital role in enhancing institutional efficiency by automating processes and reducing the reliance on paper-based systems. They deliver a more personalized and connected educational experience for both students and faculty. By improving data accuracy, accessibility, and

communication, SIS platforms help institutions operate more effectively, ensuring that academic and administrative functions run smoothly.

### 1.2 Importance of SIS in Educational Institutions

Student Information Systems (SIS) have become a foundational component in the digital transformation of educational institutions. As the volume and complexity of student-related data increase, SIS platforms offer a centralized and efficient solution for managing academic, administrative, and financial operations. These systems enhance institutional efficiency by automating routine tasks such as attendance tracking, grade processing, and fee management, thereby reducing manual errors and administrative overhead.

By automating processes like attendance tracking and grade processing, SIS significantly reduces the administrative burden. These systems also mitigate the risk of manual errors that may occur when handling large volumes of student data. This results in more accurate records and quicker response times for students and staff alike, ensuring smooth operation of academic functions.

SIS improves accessibility by allowing students, teachers, and administrators to retrieve and update information in real-time through a web-based interface. This fosters transparency and accountability, as users can view academic progress, dues, and timetables without delays. Real-time updates ensure that stakeholders always have the most current information at their fingertips, leading to more efficient decision-making and problem-solving.

By consolidating all student records into a single, secure platform, the SIS ensures data consistency and simplifies institutional reporting and analysis. As educational institutions scale and evolve, SIS platforms can adapt to new requirements with minimal restructuring. In essence, a Student Information

System is an indispensable tool that supports informed decision-making, improves communication, and elevates the overall educational experience for all stakeholders.

### 1.3 Project Objectives

The main objective of this project is to develop a centralized Student Information System that simplifies academic and administrative tasks for students, teachers, and administrators. The system provides easy access to academic details such as attendance, CGPA, arrears, timetables, and downloadable resources.

The system aims to streamline academic tasks for both students and teachers. Students can easily view their academic records, including attendance and CGPA, while teachers can manage subject allocations efficiently. By centralizing these processes, the system reduces administrative burdens and allows stakeholders to focus more on educational outcomes.

One of the key features of the project is the integration of an online payment system through Razorpay. Students can make payments directly through the platform, with automated receipt generation for transactions. This feature enhances the payment experience by providing instant confirmation and receipts, simplifying financial management for both students and the institution.

To ensure security, the system implements JWT-based authentication alongside QR code integration for secure login. These measures safeguard sensitive student information and provide a seamless, secure access experience. The system’s user interface is designed to be responsive, ensuring it works across various devices and providing an intuitive, accessible experience for all users.

## 1.4 Scope of the Project

This project covers the design and development of a web-based Student Information System tailored for educational institutions. It aims to provide students with a central platform to access and manage their academic information efficiently.

Students can view essential academic records, including attendance, CGPA, and arrears, and have the ability to download notes and circulars. Additionally, they can check their eligibility for hall tickets and manage dues through online payment systems integrated with Razorpay.

Teachers can utilize the system to manage their classes, access teaching schedules, and track student performance. Administrators have control over user roles and overall system operations, allowing for efficient management of the platform. They also oversee the management of resources such as notes and circulars, ensuring smooth functionality across departments.

The system ensures secure user authentication through JWT and QR code-based login, protecting sensitive student and administrative data. With a focus on responsiveness, the platform is designed to function seamlessly across multiple devices. The system’s scalability ensures it can adapt to the needs of growing educational institutions, while role-based access controls guarantee that only authorized users can access specific features.

# CHAPTER – 2

# LITERATURE SURVEY

## 2.1 Existing Systems and Solutions

Several Student Information Systems are currently in use across educational institutions, each offering various levels of functionality. Common platforms include open-source solutions like Fedena and ERPNext Education, as well as proprietary systems such as PowerSchool and Blackbaud. These systems generally provide essential features like student record management, attendance tracking, grade reporting, and fee collection.

Due to their wide range of features, these platforms are adopted by many institutions seeking to digitize academic and administrative processes. They often come with user-friendly interfaces, cloud hosting options, and technical support, making them attractive for deployment in large-scale institutions.

However, many of these systems are either too generalized or lack flexibility for institution-specific customizations. Some require expensive licenses and ongoing support for integration and updates. This makes it difficult for certain institutions to adapt them to local academic workflows or UI preferences, highlighting the need for more customizable and affordable alternatives.

## 2.2 Comparative Analysis of SIS Platforms

Student Information Systems vary widely in flexibility, features, cost, and user experience. Open-source platforms like Fedena offer basic modules and provide room for customization. These systems are budget-friendly and attractive to institutions with technical teams capable of managing and extending the software independently.

However, open-source platforms may lack regular updates, formal support, and advanced functionalities, making them less reliable for institutions requiring long-term scalability or strong security assurances. Their success largely depends on the availability of internal expertise and resources for maintenance.

Commercial solutions such as PowerSchool and Blackbaud come with comprehensive features, cloud hosting, and dedicated customer support. They are ideal for institutions seeking plug-and-play systems with minimal technical overhead. However, these platforms often carry high licensing costs and offer limited adaptability to specific institutional requirements.

In contrast, custom-built SIS platforms give institutions complete control over design, features, and integrations. They allow the addition of custom modules like QR code authentication, dynamic dashboards, or localized academic policies. This project exemplifies a custom-built approach that bridges the gap between rigid off-the-shelf solutions and under-supported open-source alternatives, delivering flexibility without compromising usability.

## 2.3 Research Gap and Opportunities

While many existing Student Information Systems (SIS) efficiently manage basic academic operations, they tend to fall short in areas critical to modern digital education. These include real-time data updates, modular role-specific interfaces, and seamless user experiences optimized for both desktop and mobile devices.

Most commercial or open-source platforms lack native support for features such as integrated online payment gateways, QR code-based authentication, and automated document generation (like PDF receipts). As institutions adopt more digital workflows, these limitations hinder operational efficiency and user satisfaction.

Another noticeable gap is the lack of adaptability to localized institutional workflows. Many SIS platforms follow rigid data models and structures, making it difficult to support unique academic calendars, grading systems, or fee structures without substantial customization or technical expertise.

This project aims to fill these gaps by building a fully customizable system that includes secure JWT + QR authentication, dynamic payment integration with Razorpay, real-time data handling, and responsive design tailored for students, teachers, and administrators thereby creating a future-ready solution for educational institutions.

# CHAPTER – 3

# BACKGROUND AND RELATED WORK

## 3.1 Evolution of SIS Technologies

Student Information Systems have undergone a major transformation over the past few decades. Initially, educational institutions managed student records using paper-based methods and spreadsheets, which were often disorganized, prone to human error, and time-consuming to maintain.

The introduction of desktop-based applications marked the first step toward digitization. These early systems allowed localized record management but lacked real-time accessibility, multi-user support, and integration with other systems.

As internet technologies matured, web-based SIS platforms became prevalent. These offered centralized databases, better user interfaces, and remote access enabling users to manage academic and administrative information more efficiently from any location.

Modern SIS platforms now incorporate advanced features such as cloud storage, mobile-first design, real-time data synchronization, and API-driven architecture. These innovations support dynamic academic workflows, promote collaboration, and ensure secure, scalable management of student data in today’s fast-paced educational environments.

### 3.2 Key Features and Functionalities in SIS

A modern Student Information System (SIS) encompasses a wide range of features essential for managing academic and administrative operations. Core functionalities include student profile management, attendance tracking, marks and grading systems, and timetable scheduling.

It also facilitates academic performance analysis, helping educators and administrators make data-driven decisions. The system is designed with multi-role access, ensuring that students, teachers, and administrators interact with the system through secure, role-specific interfaces tailored to their responsibilities.

Beyond the basics, additional functionalities enhance convenience and efficiency. These include digital downloads of important documents like notes, circulars, and hall tickets; online fee payment integration through gateways like Razorpay; and automated generation of PDF receipts for transactions.

To further improve accessibility and engagement, many modern SIS platforms incorporate real-time notifications, analytics dashboards, and responsive mobile compatibility. Together, these features streamline academic workflows, boost institutional productivity, and deliver an intuitive user experience.

### 3.3 Role of Automation and Integration in SIS

Automation is a key driver in the efficiency of modern Student Information Systems. It reduces manual workload and human error by handling repetitive tasks like automatic grade calculation, attendance percentage computation, and automated receipt generation for payments.

These automated features streamline administrative workflows, enabling staff to focus on more critical tasks while ensuring consistency and accuracy in academic records. Automation also helps maintain up-to-date information with minimal intervention.

Integration plays a complementary role by connecting different modules within the system. For example, payment status can be directly linked to hall ticket eligibility or result generation can depend on attendance records creating a seamless and intelligent system workflow.

Additionally, the system integrates with third-party services like Razorpay for secure online payments and QR code technology for authentication. These integrations not only expand functionality but also enhance user experience and system security, ensuring a reliable and cohesive platform for academic institutions.

# CHAPTER – 4

# TECHNOLOGY STACK

## 4.1 Overview of Technologies Used (MERN Stack)

This project is built using the MERN stack, a powerful combination of technologies that enables full-stack JavaScript development. The stack consists of MongoDB, Express.js, React, and Node.js, each of which plays a vital role in delivering a seamless user experience and efficient application performance.

React is used for developing the frontend of the system. It allows for the creation of a dynamic, interactive user interface that responds efficiently to user inputs. With React’s component-based architecture, reusable UI components are easily managed, making the frontend highly maintainable and scalable.

Node.js and Express.js are responsible for handling the backend logic, routing, and API creation. Node.js offers a non-blocking, event-driven architecture, making it well-suited for building fast and scalable backend services. Express.js simplifies the development of RESTful APIs, allowing for smooth communication between the frontend and backend components.

MongoDB is used as the NoSQL database to store user data, academic records, and transaction details. It offers high flexibility and scalability, making it ideal for managing large datasets. MongoDB's document-based structure provides fast and efficient retrieval of information, essential for maintaining data integrity and performance across various system modules.

### 4.2 Frameworks, Libraries, and Tools

This project leverages a variety of development tools and frameworks beyond the MERN stack to enhance performance, maintainability, and user experience. Each tool was carefully selected to solve specific challenges during development and to ensure a modern, scalable solution.

Redux is used to maintain a centralized and predictable state across the application, ensuring data consistency and simplified debugging. React Query complements Redux by handling asynchronous data fetching, caching, and background updates, enabling faster and smoother UI experiences.

Axios is implemented as the HTTP client for interacting with the backend REST APIs. It provides an easy-to-use and promise-based syntax for sending requests, handling errors, and integrating request interceptors, ensuring secure and efficient API communication.

TanStack Table helps manage and render large sets of tabular data dynamically. It supports advanced table features such as filtering, sorting, pagination, column resizing, and virtualization key for modules like results, payments, and student records.

Figma is used for designing clean, responsive, and user-centered interfaces. It allows real-time collaboration between stakeholders and developers and ensures that designs are consistent, scalable, and easily translatable into code components.

Tailwind CSS is the primary styling framework used to implement a mobile-first, utility-based CSS architecture. It allows for rapid development of visually appealing interfaces while maintaining full responsiveness and customization across devices.

Razorpay handles online payments securely for student dues and fees. Playwright is used for automating the generation of downloadable PDF receipts with consistent formatting, immediately after successful transactions, ensuring a smooth payment and documentation process.

Amazon S3 is used for storing downloadable files such as circulars, notes, and uploaded student documents. It provides reliable, scalable, and fast access to static assets and ensures data availability across modules that require file handling.

### 4.3 Hardware Requirements

Minimum:

• Processor: Intel i3 or equivalent

• RAM: 4 GB

• Storage: 250 GB HDD or SSD

Recommended:

• Processor: Intel i5/i7 or AMD Ryzen 5

• RAM: 8 GB or more

• Storage: 500 GB SSD

### 4.4 Software Requirements

• Operating System: Windows 10/11, Linux, or macOS

• Node.js (v18 or later)

• MongoDB Community Server

• Code Editor: Visual Studio Code

• Browser: Google Chrome or Firefox

• PDF Viewer (for receipt viewing)

These configurations support smooth development, testing, and usage of the Student Information System.

# CHAPTER – 5

# DESIGN AND ARCHITECTURE

## 5.1 System Architecture Overview

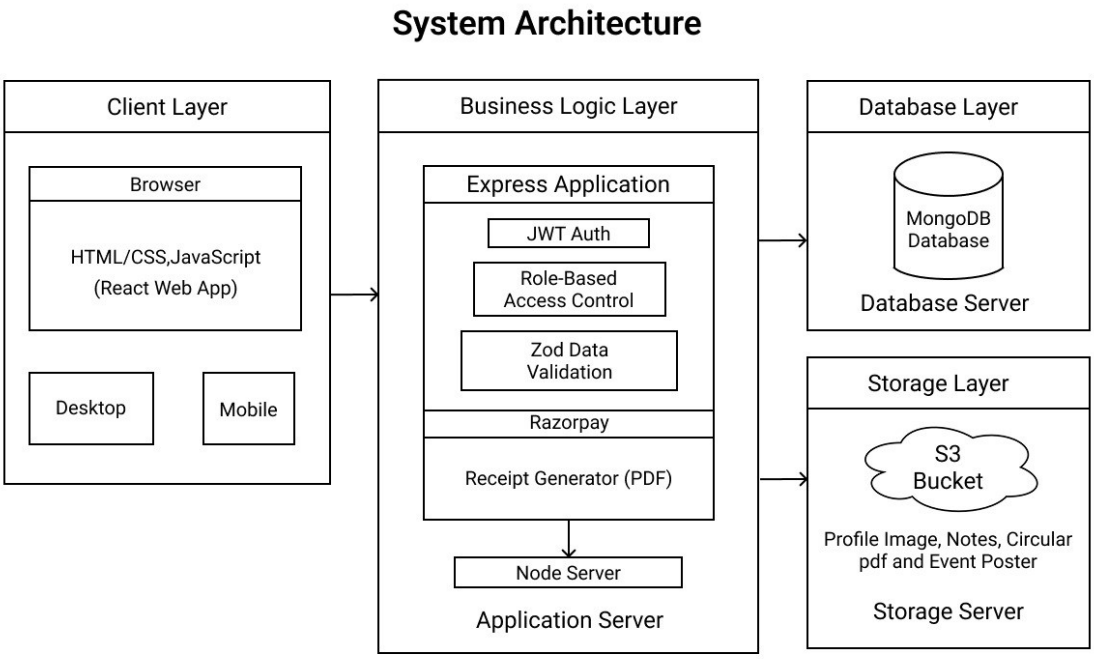



Figure 5.1: System Architecture Diagram

The following diagram provides a visual representation of the system architecture.

The Student Information System (SIS) follows a robust client-server architecture built using the MERN stack, which includes MongoDB, Express.js, React, and Node.js. This architecture ensures high scalability, secure data flow, and a seamless user experience, making it adaptable to the needs of educational institutions.

### 5.2 Client-Server Architecture with MERN Stack

At the core of the SIS is the MERN stack, a powerful combination of technologies that support full-stack JavaScript development.

MongoDB serves as the system's NoSQL database, providing flexibility in storing a variety of data types, including student profiles, academic records, attendance, grades, and financial transactions. This database supports scalability and efficient data retrieval.

Express.js, running on the server side, manages HTTP requests, routes, and middleware. It ensures that API requests are processed efficiently and that data is validated before being stored in the database.

React is used for building the frontend interface of the SIS. React's component-based architecture allows for the development of a dynamic, interactive UI that responds quickly to user input and updates data in real-time.

Node.js serves as the runtime environment for the server-side application, providing the system with non-blocking, event-driven architecture to handle concurrent requests efficiently.

These components communicate with each other via RESTful APIs, where the React frontend makes requests to the Node.js backend, which in turn processes the data and interacts with MongoDB to retrieve or store information.

### 5.3 Backend Services and Database Interaction

The Node.js and Express.js stack is responsible for the backend services, where the business logic of the system is handled. API requests are routed through Express.js, which processes them and interacts with MongoDB to manage data storage and retrieval.

MongoDB’s document-based structure allows the system to store diverse types of data, such as student attendance records, academic results, and fee details, in a scalable manner. Data retrieval and modification operations are optimized through MongoDB’s flexible querying capabilities.

The backend is designed to be modular, meaning each service (e.g., payment processing, authentication, user management) operates independently, making the system easier to maintain and scale.

### 5.4 Authentication and Security

Security is a key aspect of the SIS, and to protect user data, the system implements JWT-based authentication. When a user logs in (whether a student, teacher, or admin), a JWT token is generated. This token is then included in each subsequent request to verify the user’s identity and ensure they have the correct permissions to access specific resources.

In addition to JWT authentication, the system also integrates QR code authentication for enhanced security. QR codes are particularly useful for mobile users, providing a secure and easy way to authenticate without exposing login credentials. This multi-layered authentication process ensures that users are securely logged into the system.

To extend functionality and improve user experience, the SIS integrates with third-party services.

Razorpay is incorporated for handling online payment transactions. This allows students to make fee payments through a secure payment gateway. Razorpay supports multiple payment methods, including credit/debit cards, UPI, and wallets, ensuring a seamless and secure payment experience.

Playwright is used to generate automated PDF receipts after each successful payment. These receipts contain transaction details and can be downloaded by users for their records. Playwright ensures that PDF generation is fast, accurate, and formatted consistently across all receipts.

The system architecture is designed with modularity in mind, meaning that each component operates independently while maintaining seamless integration with the other parts of the system. This modular approach ensures that the system is flexible and scalable, allowing for future enhancements and easy integration of new features as needed.

## 5.5 Data Flow and Process Flow Diagrams

The Data Flow Diagram (DFD) illustrates how data moves within the Student Information System (SIS). It provides a visual representation of how information flows between the various components of the system and the external entities involved in the process.

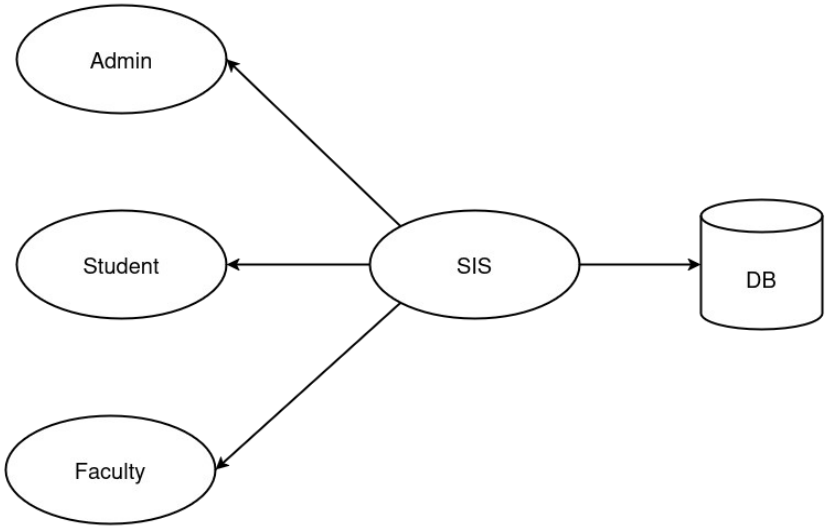



Figure 5.5: Data Flow Diagram

The DFD outlines the interaction between the system components and external entities such as students, teachers, and administrators. It highlights how data is exchanged, ensuring the system can efficiently handle requests and responses while maintaining data integrity across different processes.

The Process Flow Diagram illustrates key operations within the system, such as login, viewing academic records, downloading resources, and making payments. It outlines the sequence of steps involved in each module, providing a clear view of the system's functionality.

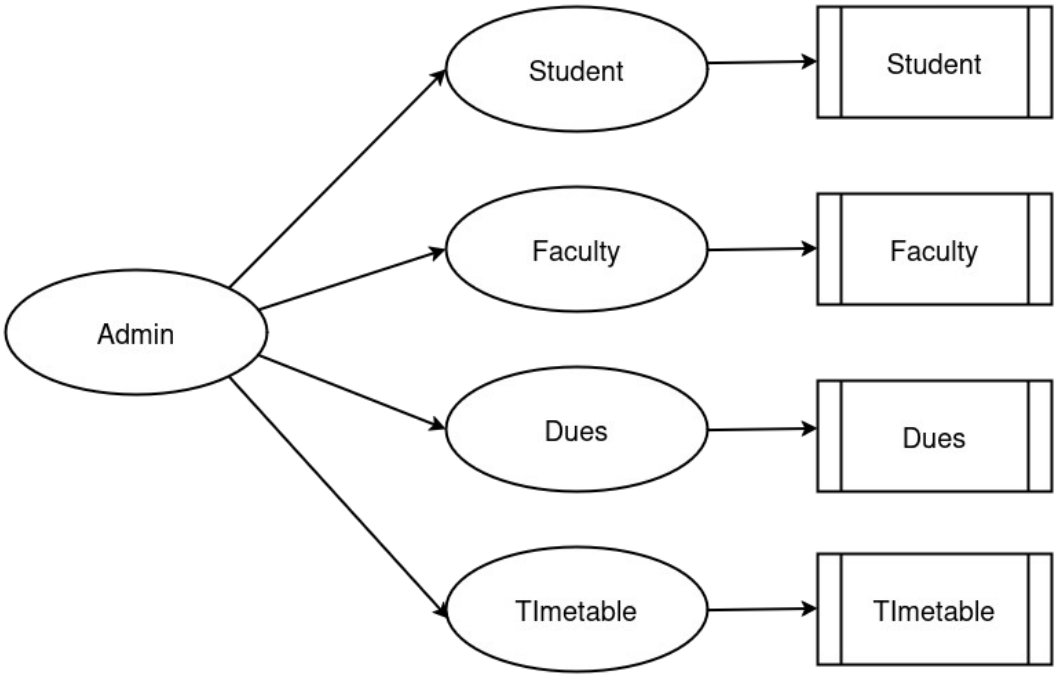



Figure 5.5: Admin Module – Process Flow Diagram

Each module interacts through defined steps, ensuring smooth navigation and system response. These diagrams play a crucial role in visualizing the logic behind each operation, ensuring clarity in both the design and development processes.

### 5.6 User Interface and Navigation Design

The user interface was designed in Figma with a focus on simplicity, responsiveness, and accessibility. The layout was structured to meet the needs of various user roles students, teachers, and admins providing each with a tailored dashboard to access their relevant features.

Key modules, including attendance, CGPA, dues, timetable, and resources, are organized in a sidebar for easy and efficient navigation. This design helps users quickly access the information they need without unnecessary clutter or confusion.

Students can seamlessly switch between different features, such as viewing results, downloading notes, and checking their fee status. Teachers can easily access subject allocations and upload study materials. Admins have a centralized interface to manage student and teacher records through structured forms.

The interface is designed with clarity and ease of use in mind, ensuring that users can interact with the system intuitively. With an emphasis on minimalistic design, unnecessary steps are avoided, resulting in a faster and more efficient user experience.

The intuitive design ensures smooth navigation across devices and screen sizes, maintaining a consistent user experience.

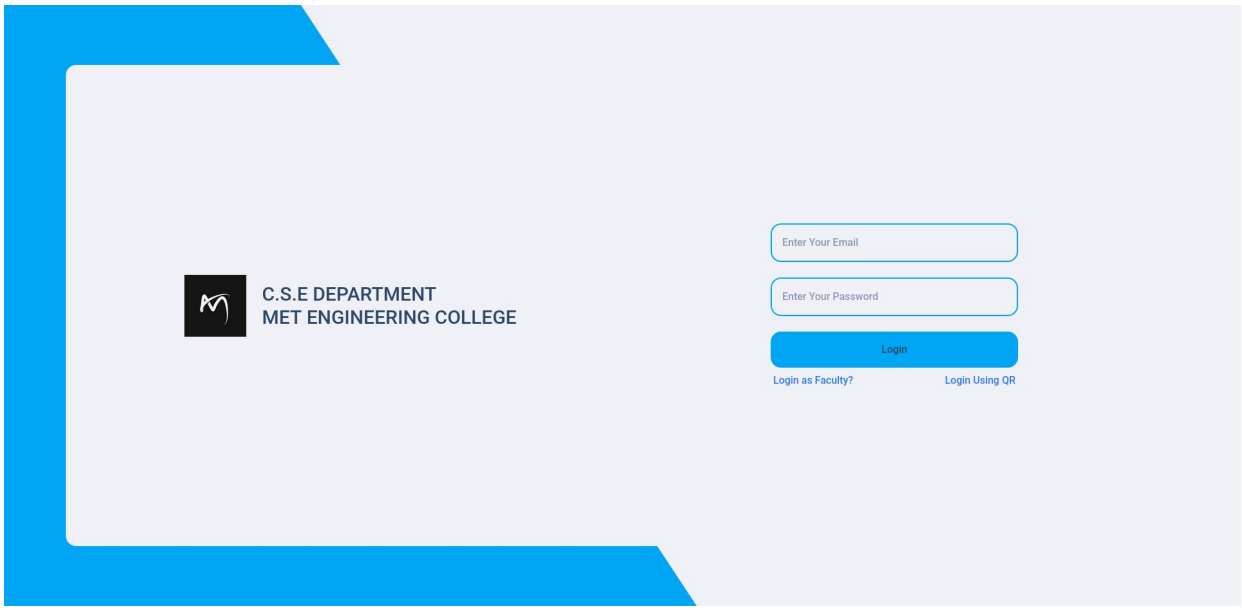



Figure 5.6: Login Page

The login page is the entry point for all users, providing secure authentication. Once logged in, users are directed to their specific dashboards.

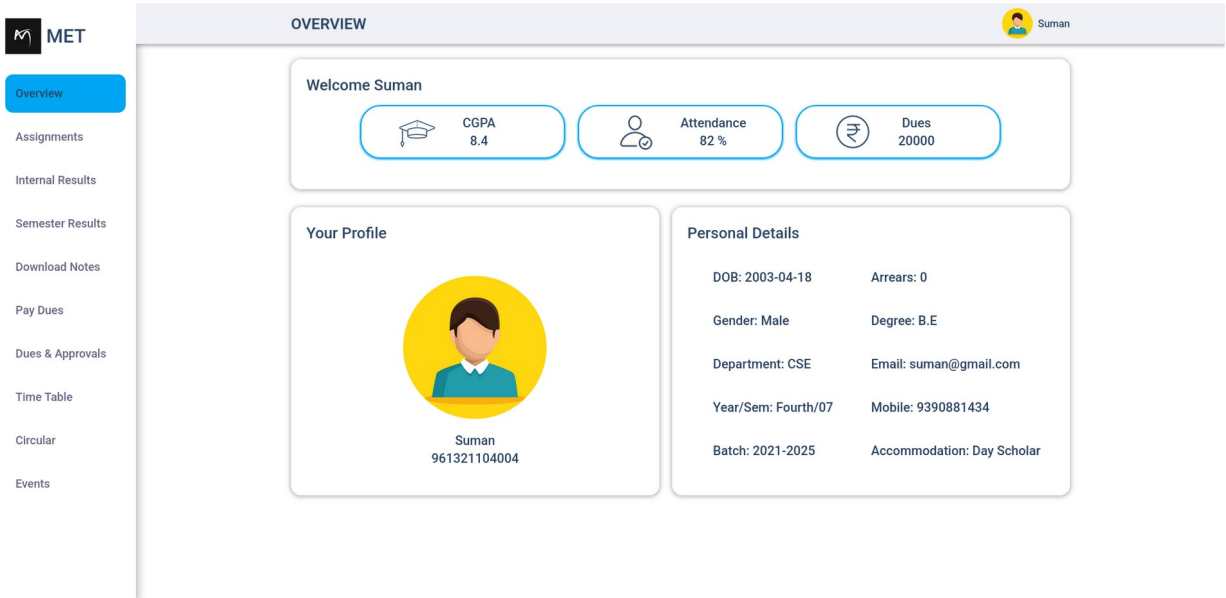



Figure 5.6: Student Overview Page

The Student Overview page displays key bio-data, attendance, CGPA, dues balance, and more. The sidebar offers easy navigation to other sections, creating an organized and user-friendly interface.

# CHAPTER – 6

# MODULE IMPLEMENTATION

## 6.1 Student Details Management

The Student Details Management module allows administrators to create, update, view, and delete student records efficiently. It stores essential student information including name, registration number, department, semester, contact details, and login credentials. This centralization of student data ensures that it is easy to access and manage, contributing to an organized system that supports the daily operations of educational institutions.

This module ensures data accuracy and consistency, serving as the foundation for other system functionalities like attendance tracking, result generation, dues management, and timetable access. By maintaining clean, accurate student records, it enables the system to generate real-time reports and analytics. It also plays a critical role in providing personalized services to students, ensuring their academic and administrative details are always up to date.

It supports role-based control to maintain data privacy and prevent unauthorized modifications. The system ensures that only authorized personnel, such as administrators or staff members, have access to sensitive student data. This ensures a high level of security and prevents any potential data breaches, which is essential in protecting the privacy and confidentiality of students.

## 6.2 Marks and Grading Module

The Marks and Grading Module enables faculty and administrators to record and manage students' academic performance across different types of

assessments such as mid-term exams, assignments, lab work, and semester-end exams. It provides a structured interface for entering marks and supports bulk uploads, making it suitable for handling large sets of academic data efficiently. The module allows flexibility in setting up grading policies tailored to the institution's requirements, whether using raw scores, letter grades, or CGPA formats.

Students have access to their marks and grades in real time through their respective dashboards, encouraging transparency and academic accountability. The module can be linked with other parts of the system such as attendance tracking or dues management so that institutions can optionally factor in attendance records or payment status when calculating final results. This integration ensures a holistic and accurate evaluation of student performance while minimizing administrative workload.

### 6.3 Fee Management and Payment Integration

The Fee Management and Payment Integration module is responsible for tracking, organizing, and processing all types of student financial transactions, such as tuition fees, exam fees, and miscellaneous charges. Administrators can define or update fee structures based on academic year, course, or category, and record payments as either online or offline. The system maintains a clear log of due amounts, paid transactions, and pending balances, making it easier to monitor overall financial records.

Students can conveniently access their dues via their dashboards and complete payments through the integrated Razorpay payment gateway. The module ensures a secure and user-friendly payment experience while offering real-time transaction updates. This automation reduces the need for manual bookkeeping and enhances

financial transparency between students and institutions. Additionally, it supports automated receipt generation after successful payments, further streamlining administrative workflows.

### 6.4 Receipt Generation

The Receipt Generation module is designed to automatically create digital payment receipts for students after successful fee transactions. This system eliminates the need for manual receipt creation and ensures that students receive instant confirmation and documentation of their payments. Each receipt contains detailed information including the student’s name, registration number, payment amount, date, payment method, and a unique transaction ID.

To enhance presentation and professionalism, receipts are generated in a structured and printable PDF format using Playwright, a powerful browser automation tool. This ensures that all visual elements such as logos, headers, tables, and styling are consistently applied, making the receipts easy to read and suitable for official use.

Receipts are generated dynamically based on real-time transaction data and are made available for download immediately after payment. The system also maintains a history of all past receipts for future reference, allowing students and administrators to retrieve them whenever required. This feature greatly improves user convenience and record-keeping efficiency.

The module plays a key role in supporting financial transparency and accountability within the institution. By automating the entire process and integrating it with the fee management system, it reduces administrative workload, avoids human error, and ensures timely delivery of payment proofs to students.

### 6.4 QR Code Authentication

QR code authentication was introduced to enhance the security and user experience within the system. This feature allows users to log in by scanning a QR code displayed on the login page. The QR code represents a JWT (JSON Web Token), which is converted into a QR format for easy scanning. This solution adds an extra layer of security, as the token is tamper-proof and can only be used once for authentication.

The JWT token is securely converted into a QR code format, ensuring that the token cannot be easily manipulated during the authentication process. This reduces the risk of unauthorized access and ensures that only valid tokens are accepted. The QR code authentication method was designed to offer a fast, secure, and seamless login process for users.

To further enhance security, the JWT token is stored in the database in a hashed format. This prevents sensitive information from being exposed in the event of a data breach. The system ensures that tokens are hashed before storing them, making it virtually impossible for attackers to retrieve or reverse-engineer the original token from the database.

This tamper-proof approach to authentication strengthens the overall security of the system by ensuring that tokens are both difficult to intercept and verify. By combining QR code authentication with JWT and hashing, the system provides a robust, secure login mechanism that is easy for users to implement while protecting sensitive information.

# CHAPTER – 7

# TESTING AND VALIDATION

## 7.1 Testing Strategy and Tools Used

The system underwent module-wise testing using both manual and automated approaches to ensure functionality, performance, and security. Unit testing was performed for backend APIs, while integration and UI testing validated end-to-end workflows. The testing process was crucial to verify the correctness and reliability of each module, ensuring they operated as expected.

Tools like Postman were used for API validation, and browser developer tools aided in front-end debugging. Regular testing cycles helped identify and resolve issues early, ensuring system stability before deployment. This proactive approach to testing minimized bugs and errors, contributing to a smoother user experience and a more secure, reliable system upon launch.

## 7.2 Test Cases for Each Module

Each core module was tested with defined test cases covering both valid and invalid inputs to verify expected outcomes. The Student Details module was tested for CRUD operations, ensuring accurate creation, reading, updating, and deletion of student data. The Marks module was checked for correct data entry and retrieval, ensuring the system processed grades and academic records without errors.

Fee management and receipt generation were validated using sample payment scenarios, ensuring successful transactions and receipt availability. Authentication flows, including JWT and QR login, were tested for security and access control

across different user roles, ensuring that only authorized users could access specific functionalities.

### 7.3 Security and Data Integrity Testing

Security testing focused on validating JWT-based authentication, role-based access, and QR code login. The authentication mechanism was rigorously tested to ensure that unauthorized users could not gain access to sensitive parts of the system. Every login attempt was checked for potential vulnerabilities, including the security of the QR code authentication method, to ensure user data protection.

Role-based access was tested to verify that users with different privileges could only access the features and data relevant to their roles. Administrators, teachers, and students were each assigned specific access levels, and their access rights were validated during the testing phase. This ensured that data confidentiality and user privacy were maintained.

Data integrity was ensured through input validation at both the backend and frontend. Any data entered into the system was validated to ensure it adhered to expected formats and values. This helped prevent invalid or malicious data from entering the system, preserving the accuracy and reliability of student records.

Consistent database operations across all modules were also validated. This testing ensured that data was correctly written to and retrieved from the database without discrepancies or errors. All interactions with the database, including read, write, and update operations, were tested to ensure that the system maintained data integrity and accuracy throughout its lifecycle.

# CHAPTER – 8

# RESULTS AND ANALYSIS

## 8.1 Performance Metrics

The system was evaluated based on response time, load handling, and database efficiency. Backend APIs consistently returned responses within acceptable timeframes under normal user loads, ensuring that users did not experience delays while interacting with the system. These APIs were optimized to handle typical traffic efficiently, contributing to a smooth user experience.

Frontend rendering remained smooth due to optimized React components and efficient state management with Redux and React Query. The use of these technologies ensured that the user interface remained responsive even when multiple components were interacting with the backend. React Query facilitated the management of server-state data, while Redux helped maintain consistent client-side state, reducing unnecessary re-renders.

Database queries were indexed for faster retrieval, especially in high-traffic modules like results and fee management. By optimizing queries and indexing commonly accessed data, the system ensured that it could handle multiple requests simultaneously without significant performance degradation. Overall, the system demonstrated stable performance and responsiveness during testing, meeting the expected performance benchmarks.

### 8.2 User Feedback and Usability

Initial user feedback from students, teachers, and administrators highlighted the system's ease of use and clean interface. Users appreciated the organized navigation, which made it easy to find essential information quickly. The smooth login process and intuitive design contributed to a positive user experience, ensuring that all users could access their respective data effortlessly.

Features like real-time marks viewing, downloadable receipts, and the integrated timetable were well-received. Users found these functionalities particularly useful, as they provided immediate access to important academic and financial information. Suggestions mainly focused on minor UI improvements and expanding certain features, confirming the system’s overall usability and effectiveness in meeting user needs.

### 8.3 Data Accuracy and Reporting Quality

The system maintained high data accuracy through strict validation checks and role-based data entry. All academic records, dues, and results were cross-verified during testing to ensure correctness. This rigorous validation process helped prevent data inconsistencies and ensured that the information presented to users was accurate and up-to-date.

Reports generated for marks, payments, and attendance reflected real-time and consistent data, aiding in transparent evaluation. The centralized data structure minimized duplication and errors, ensuring reliable output across all modules. This approach enhanced the quality of reports, making them trustworthy and valuable for both students and administrators.

### 8.4 Overall System Evaluation

The Student Information System demonstrated robust functionality, efficient performance, and ease of use throughout testing. All core features student record management, marks tracking, fee processing, and receipt generation operated seamlessly, providing users with reliable access to their academic and financial data. These essential features worked together smoothly, supporting the system’s overall effectiveness.

The integration of payment gateways, secure authentication, and real-time data updates contributed to a comprehensive, user-friendly experience. The payment system, powered by Razorpay, ensured secure and reliable financial transactions, while QR code authentication and JWT-based security measures protected user data. Real-time updates to academic records, attendance, and fee status ensured that students and staff had the most accurate information available.

Feedback from users affirmed the system's ability to streamline academic and administrative tasks effectively. Users appreciated the simplicity of accessing essential information and interacting with various features. Performance and security measures met the project’s objectives, confirming that the system is both efficient and secure for everyday use.

# CHAPTER – 9

# CHALLENGES AND LIMITATIONS

## 9.1 Technical Challenges

During development, managing the monorepo structure with Turborepo presented initial complexities in package linking and environment configuration. Ensuring that all packages were properly linked and that the environment settings were correctly configured across different services took significant effort. These challenges were mitigated with careful setup and configuration of Turborepo to streamline development.

Ensuring secure session handling and role-based access across different user types required careful design. With multiple user roles, the authentication system had to ensure that each role had the appropriate permissions to access different resources. Integrating Razorpay and generating dynamic PDF receipts using Playwright also involved handling asynchronous workflows and edge cases. Optimizing database queries for performance, especially in high-usage modules like the fee and result sections, was another key challenge that was addressed through indexing and caching.

## 9.2 Operational and User-Related Issues

Some users initially faced difficulty navigating certain modules due to unfamiliarity with the interface. Despite the clean design, certain areas of the system required additional guidance for first-time users. These navigation challenges were identified through user feedback, and adjustments were made to improve accessibility.

Inconsistent data entry formats from different administrators caused occasional issues in the result and fee sections. To address this, data validation and standardized formats were introduced. Handling payment failures and retry scenarios also required additional user guidance, which was provided through clearer error messages and step-by-step instructions for students and staff.

### 9.3 Limitations of the Current System

The system currently lacks mobile app support, limiting accessibility on smaller devices. While the web-based interface is responsive, users who prefer mobile access may find it less convenient. This limitation could be addressed in future iterations by developing dedicated mobile apps or enhancing mobile responsiveness further.

It also depends on stable internet connectivity, with no offline access or sync features. Users in areas with intermittent internet access might face difficulties when using the system. Advanced analytics and reporting tools for academic performance are minimal, and the system could benefit from additional insights and data visualization features. Additionally, manual data entry for marks and results can be time-consuming and may benefit from future automation or import features to improve efficiency.

# CONCLUSION

The Student Information System offers a complete solution to common academic and administrative challenges. Built with modern web technologies and a monorepo architecture, it provides a unified, user-friendly platform for students, teachers, and administrators to manage essential information efficiently

Key features were thoughtfully implemented to ensure reliability, functionality, and security. Students can independently access their academic progress, attendance, dues, and timetable. Teachers manage class schedules and subjects efficiently, while administrators handle records, results, payments, and data integrity with ease.

Advanced features like QR code authentication enhance security and ease of access, while Razorpay integration streamlines fee payments. Automated PDF receipt generation using Playwright ensures consistent documentation, and the Figma-designed responsive interface offers a smooth user experience across devices.

The testing phase confirmed the system's robustness, highlighting its performance stability, secure access control, and accurate data reporting. User feedback further validated its intuitive design and real-world applicability.

In conclusion, the Student Information System fulfills its objectives and sets the stage for future digital growth in education. Its modular structure enables easy expansion, simplifying academic and administrative tasks for smarter campus management.

## FUTURE ENHANCEMENT

While the current system fulfills the essential requirements of a Student Information System, several enhancements can be introduced to improve its functionality, accessibility, and user experience.

One of the primary future goals is to develop a dedicated mobile application to complement the web platform, offering students and teachers easier access on the go. Offline access with data synchronization features can be integrated to support users in low-connectivity environments.

Automation in data entry, such as bulk upload of marks and attendance via CSV or Excel files, can save time and reduce human errors. Additionally, advanced analytics and dashboards can be implemented for academic insights, performance trends, and decision-making support.

Another promising enhancement is voice control integration. By incorporating tools like Alan AI or Speechly, users could navigate and interact with the system through voice commands, making the platform more accessible, especially for differently-abled users.

Furthermore, support for multilingual interfaces, chatbot-based help systems, and integration with institutional ERP systems can significantly extend the system’s capabilities. These enhancements will make the Student Information System not only more powerful but also more inclusive and intelligent.

# APPENDICES

This section contains supporting materials that offer additional context to the system's design and implementation.

## Screenshots of Student Form

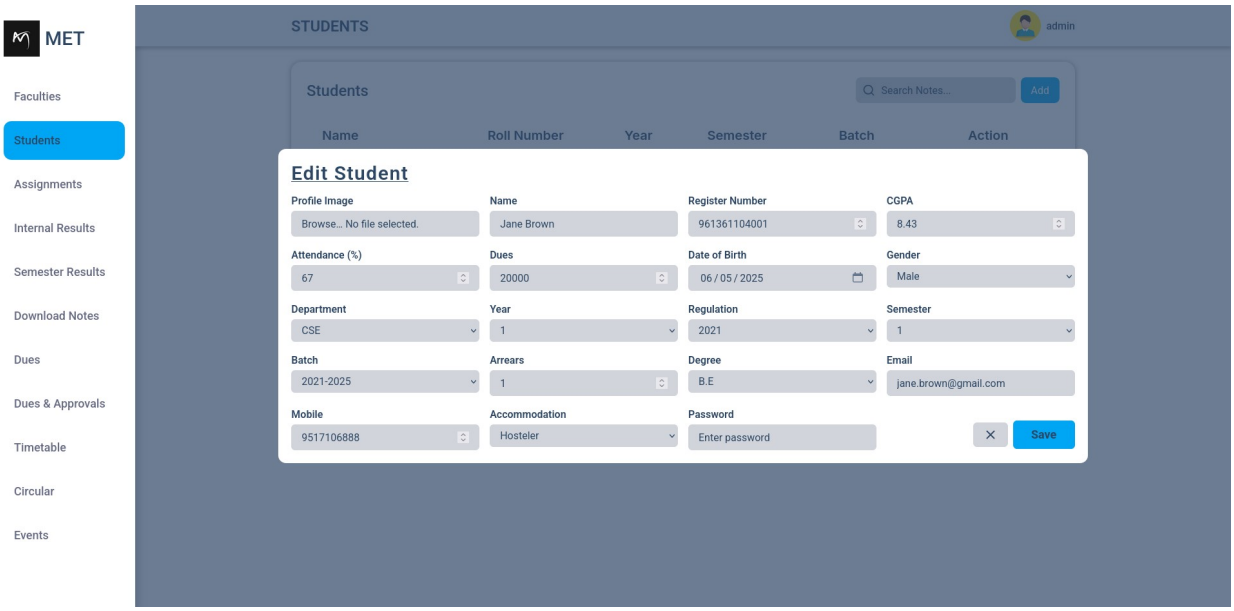



Figure : Student Details Form

## Regulation Info Schema Structure

```
import { Schema, model } from 'mongoose';

const SubjectSchema = new Schema(
  {
    code: { type: String, required: true },
    name: { type: String, required: true },
  },
```

```
    { _id: false }
);
const SemesterSchema = new Schema(
    {
        subjects: [SubjectSchema],
        laboratory: [SubjectSchema],
    },
    { _id: false }
);
const RegulationInfoSchema = new Schema({
    regulation: { type: String, required: true, unique: true },
    semesters: {
        one: SemesterSchema,
        two: SemesterSchema,
        three: SemesterSchema,
        four: SemesterSchema,
        five: SemesterSchema,
        six: SemesterSchema,
        seven: SemesterSchema,
        eight: SemesterSchema,
    },
});
```

**Internal Result Schema Structure**

```
const InternalEntry = new Schema(
    {
        code: { type: String, required: [true, 'please enter subject code'] },
        name: { type: String, required: [true, 'please enter subject code'] },
        status: { type: Boolean, default: false },
        mark: { type: Number, default: 0 },
    },
    { _id: false }
);
const InternalResultSchema = new Schema({
    registerNo: { type: Number, required: [true, 'please enter register no'] },
    name: { type: String, required: true },
    year: { type: Number, required: [true, 'please enter a year'] },
    results: {
        one: [InternalEntry],
        two: [InternalEntry],
        three: [InternalEntry],
        four: [InternalEntry],
    },
});
const InternalResultModel = model('InternalMark', InternalResultSchema);
```

**Zod Validation**

```
import { ErrorHandler } from '@utils';
import type { Request, Response, NextFunction } from 'express';
import { ZodSchema } from 'zod';


export const validate =
   <T>(schema: ZodSchema<T>) =>
   (request: Request, response: Response, next: NextFunction) => {
      const result = schema.safeParse(request.body);
      if (!result.success) {
         const formattedError = result.error.format();
            const zodError = Object.entries(formattedError).reduce<Record<string, string>>((acc, [key, value]) => {
            if (key === '_errors') return acc;
            acc[key] = (value as any)._errors[0];
            return acc;
         }, {});
                return next(new ErrorHandler(400, 'validation failed', 'ZodError', zodError));
      }
      next();
   };
```

## Screenshots of Key Interfaces

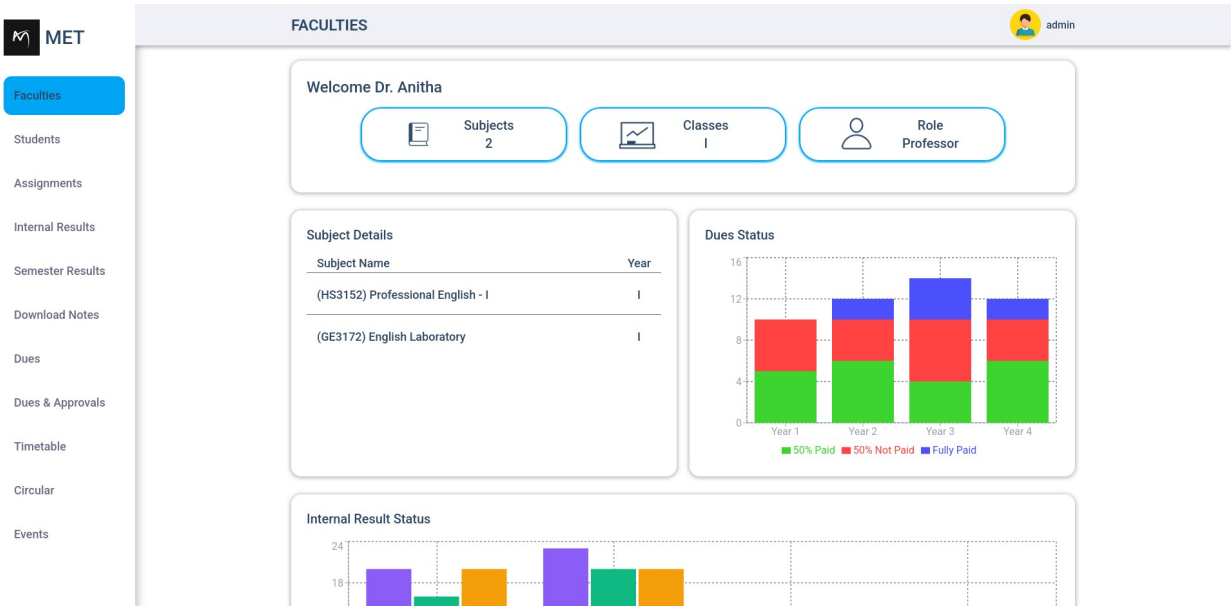



Figure : Faculty Dashboard

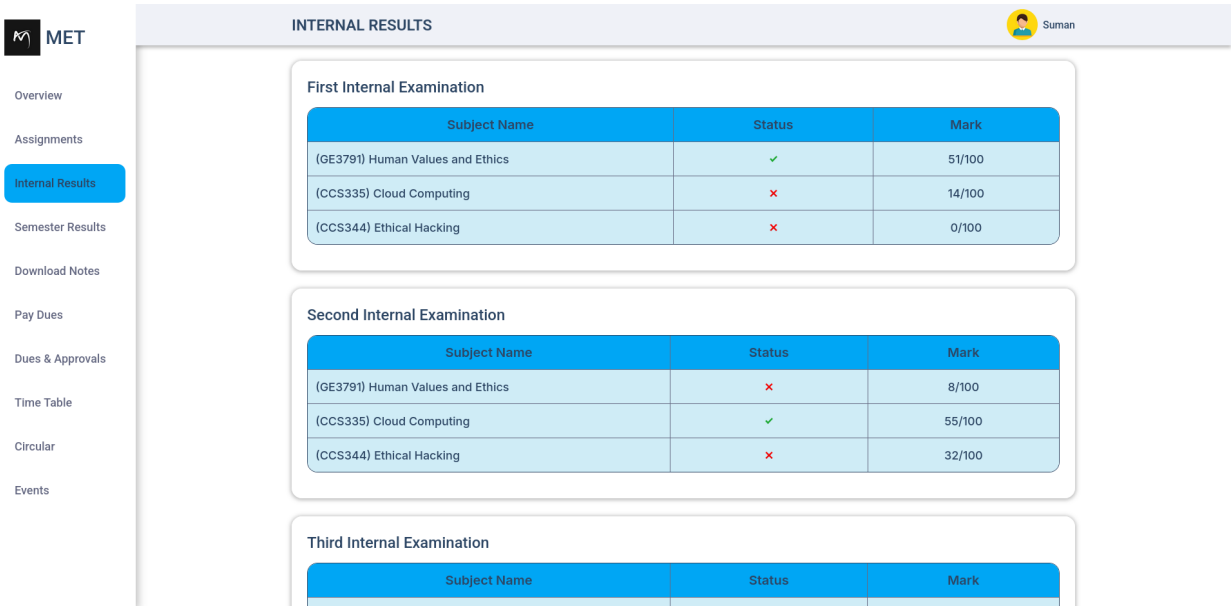



Figure : Internal Result Page

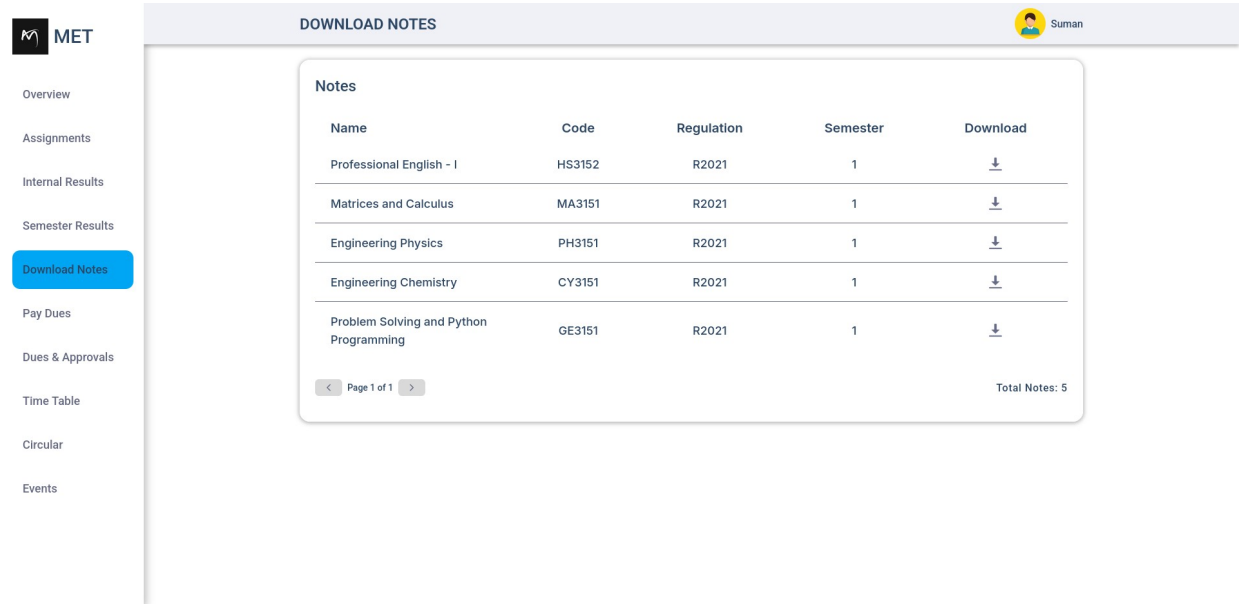



Figure : Download Notes Page

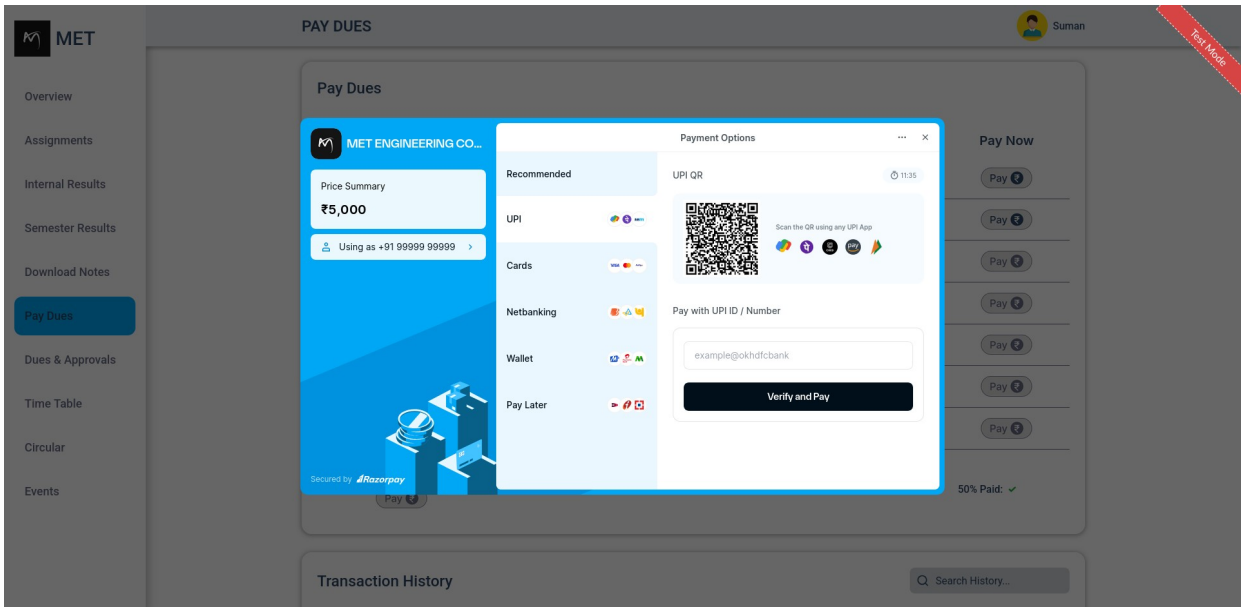



Figure : Payment Page




Figure : Dues And Approvals Page

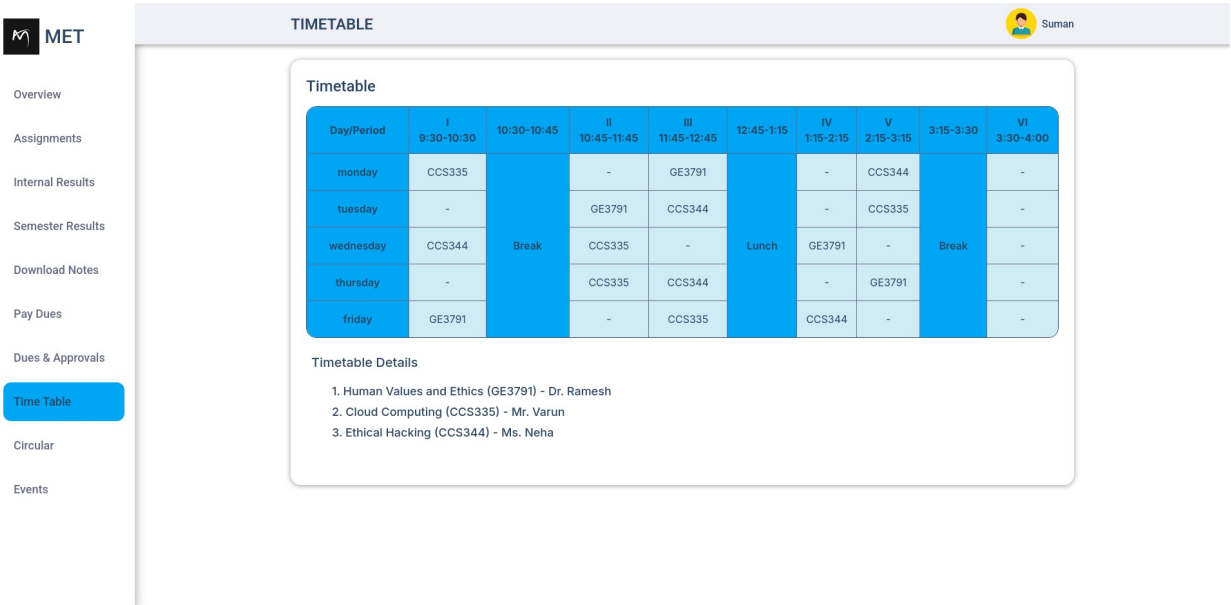



Figure : Timetable Page

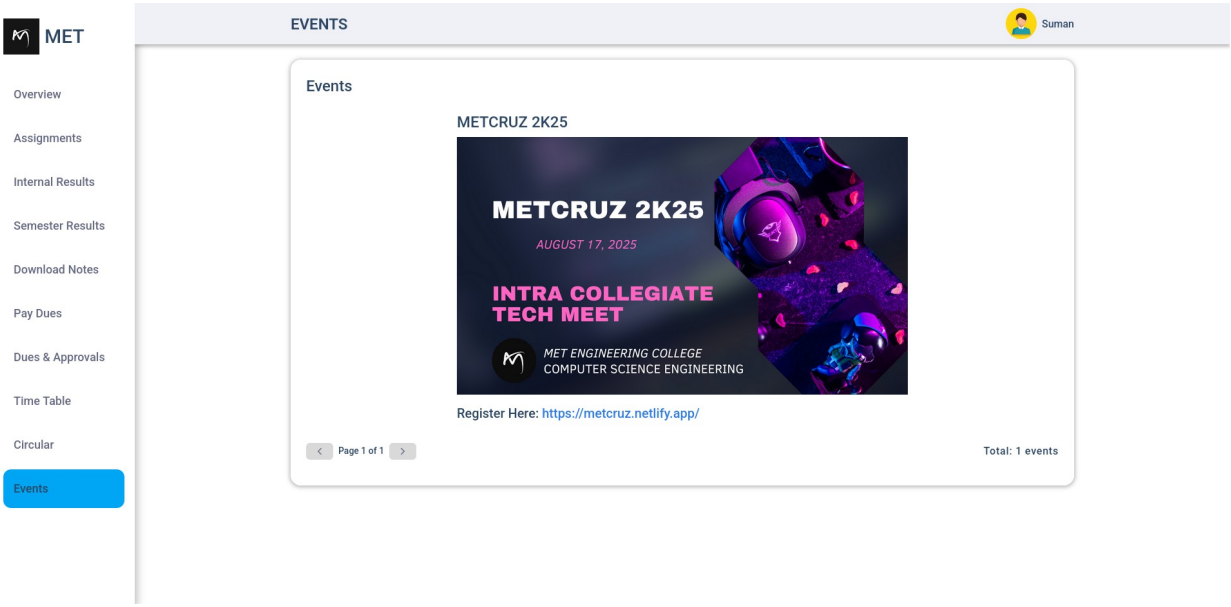



Figure : Event Page

**Sample PDF Receipt Generated using Playwright**




Figure : Payment Receipt

## REFERENCES


1. Mern Stack Documentation – [MongoDB, Express, React, Node.js]

   https://www.mongodb.com/mern-stack

2. Razorpay Payment Gateway Integration Guide

   https://razorpay.com/docs/

3. Playwright for PDF Generation – Microsoft Docs

   https://playwright.dev/

4. JWT Authentication – JSON Web Tokens

   https://jwt.io/introduction

5. Figma UI/UX Design Resources

   https://www.figma.com/

6. Alan AI – Voice AI Platform

   https://alan.app/

7. Speechly – Voice Interface SDK

   https://speechly.com/